

Marathi Machine Translation via Bodhan Fine-Tune

AI4Bharat / AI Research Engineer take-home — English → Marathi (eng_Latn → mar_Deva)

Source code (GitHub):

<https://github.com/Kaustubh-Rathi/marathi-mt-bodhan>

Artifacts, checkpoints & logs (Google Drive):

https://drive.google.com/drive/folders/1Abt7fldMDI_4pjTa3ayDbLI1dqDZSUTD

Primary model: bodhan-ai/indic-translate (Gemma-4 E4B, 8B) fine-tuned with QLoRA

Training data: ai4bharat/samanantar (config `mr`)

Evaluation: ai4bharat/IN22-Gen (test) + facebook/flores (devtest) — both held out, never trained on

Run results (live artifacts)

All three sessions trained end-to-end on Kaggle T4s and every checkpoint was mirrored to Drive. The numbers below are read directly from `reports/`, which is synced from the runs themselves.

Training runs

Session	Method	Steps	Status	GPU
A - Bodhan QLoRA (primary)	qlora	16		-
B - IndicTrans2 LoRA	lora-seq2seq	16		-
C - Ablation	qlora	16		-

Evaluation (Session B, held-out, never trained on)

Test set	BLEU	chrF2	chrF2++
IN22-Gen (test, n=1024)	20.91	54.10	49.82
FLORES devtest (n=1012)	19.49	54.90	50.69

APPROACH

The graded narrative: choices, rationale, gotchas — source: APPROACH.md

APPROACH — Marathi MT via Bodhan Fine-Tune

Results status: runs pending. Every metrics/result slot is a marked `TODO(results)` placeholder to be filled after Kaggle sessions A/B/C complete. Nothing below invents numbers.

1. Goal + constraints

- **Assignment:** AI4Bharat / AI Research Engineer take-home. Pick **one** Bodhan model,

fine-tune on **Marathi**. Deliverables: **GitHub repo**, **Google Drive artifacts**, **detailed docs**.

- **Scored on:** end-to-end run, code structure, effort/judgement/problem-solving.

NOT scored on metrics. So this doc optimizes for reviewer-verifiable decisions, not SOTA claims.

- **Hard constraints:** Kaggle GPUs only (3 sessions/accounts × 30h/week, 12h per session);

gated HF assets; no budget for hosted tracking (no MLflow server, no W&B paid tier); Bodhan 8B must fit on a T4/P100 via QLoRA.

2. Options considered / rejected

2a. Base model: IndicTrans2-200M vs Bodhan 8B (bodhan-ai/indic-translate)

Option	Pros	Cons	Verdict
ai4bharat/indictrans2-en-indic-dist-200M (gated auto, 200M)	Trains anywhere incl. T4; mature IndicTransToolkit; native eng_Latn source matches the Samanantar en→mr train direction	Capacity ceiling on fluency; Samanantar WAS in IndicTrans2 pretraining (scores partly re-measure memorization); needs a one-time gated accept	Session B — guarantees a complete end-to-end story whatever happens on A
bodhan-ai/indic-translate (Gemma-4 E4B, 8B, gated)	Assignment-aligned (Bodhan pick); strongest Marathi prior; instruction-tuned chat template	8B needs 4-bit QLoRA + careful Gemma-4 handling; gated + share-alike license	Primary (Session A) — where the effort goes

2b. Fine-tune stack: Unsloth vs TRL+PEFT vs Axolotl

Option	Pros	Cons	Verdict
Unsloth	Fastest kernels, low VRAM	Opaque patched kernels; Gemma-4 support lag; harder to attribute each hyperparameter	Rejected — not used in any session (only a commented optional line in requirements.txt); Session C is a Bodhan QLoRA ablation (r 32/64→16/32, lr 1e-4→5e-5, seq 1024→512, steps 1000→600)
TRL + PEFT (+ BitsAndBytes NF4)	Transparent SFTTrainer / LoraConfig ; public Arushhh recipe maps 1:1 onto it; auditable	Slower than Unsloth	Chosen for Session A
Axolotl	Great YAML-driven sweeps	Extra abstraction over failures we needed to see raw (ClippableLinear, mm columns)	Rejected — debugging surface too indirect for Gemma-4 gotchas

2c. Tracking: MLflow vs W&B vs TensorBoard + CSV

Option	Pros	Cons	Verdict
MLflow server	Rich registry	Needs a hosted server we don't have; overkill for 3 runs	Rejected
W&B	Nice dashboards	Account/egress friction across 3 Kaggle accounts; external dependency	Rejected
TensorBoard + reports/experiments.csv + metrics.json + Drive sync	Zero infra; per-run TB logs (gdrive:mr-mt-edu-2026/<run>); CSV is the human index	Manual figure assembly	Chosen

2d. Compute: single vs multi-account Kaggle

- Single account = 30h/week, one failure kills the story.

- **Chosen: 3 sessions/accounts** — A = Bodhan 8B QLoRA (Acct1), B = IndicTrans2

en-indic-dist-200M (Acct2), C = ablation/demo (Acct3). Each keeps every checkpoint (every 100 steps) and syncs them to Drive, so a 12h session timeout costs at most one checkpoint interval. Ethical note: three genuine separate accounts used within stated quotas; no quota circumvention beyond documented limits.

3. Model choice + why

- **Primary:** `bodhan-ai/indic-translate`. It is the assignment's Bodhan pick for Marathi,

Gemma-4 E4B 8B instruction-tuned for Indic translation. QLoRA (4-bit NF4, all-linear LoRA) is the only way 8B fits Kaggle GPUs, and the public Arushhh recipe gives a proven configuration to mirror rather than re-search under a 30h budget.

- **Session B:** `ai4bharat/indictrans2-en-indic-dist-200M` + **IndicTransToolkit**

(`eng_Latn->mar_Deva`). Gated `auto` (one-time accept), 200M params, trains on any Kaggle GPU. Guarantees submittable artifacts + metrics even if Session A stalls on VRAM or gating.

- **Attribution (license compliance):** outputs derived from `indic-translate` carry

"Built with indic-translate from Bodhan AI / AI4Bharat" and Indic Open Model License v1.0 share-alike terms.

4. Dataset choice + not-in-training rationale + decontamination

- ****Train:** `ai4bharat/samanantar`, config `mr` (OPEN, English→Marathi,

CC-BY-NC-4.0), direction `eng_Latn->mar_Deva`. **This is a pivot: the preferred set was `coild-aikosh/Education_v2` (education-domain HIN-MAR, CC-BY-4.0) — chosen because it was** unlikely to have been in Bodhan/IndicTrans2 pretraining**, unlike BPCC/Samanantar which are near-certain pretraining ingredients — but COILD is manual-gated and access was denied, so it is NOT used. Honest caveat: Samanantar (via BPCC) WAS used in IndicTrans2 pretraining, so Session B scores partly re-measure memorization; Session A (Bodhan) is less affected.

- ****Eval** (held out, NEVER trained on): `ai4bharat/IN22-Gen` (config `null/`

`default`, split `test`) + `facebook/flores` (config `eng_Latn-mar_Deva`, split `devtest`), direction `eng_Latn->mar_Deva`.** General-domain benchmarks, disjoint from the Samanantar train slice — so gains must come from transfer, not leakage.

- **Decontamination gates** (full table in `reports/LEAKAGE_CHECKLIST.md`, L1..L10):

pair-hash blocklist of test sets vs train; exact + src-only near-dup filtering (threshold 0.9, see `configs/base.yaml` `prepare_near_dup_threshold`); length filters (1–100 words); date/domain provenance check; no test files inside train dirs; normalize-before-score. Eval-decoding config: 256 new tokens, beam 5.

- Splits: `max_train` 8000 / `max_dev` 1000 (`configs/base.yaml`), seed 42, JSONL rows

`{src, tgt, src_lang, tgt_lang, domain}` per `docs/SPEC.md`.

5. Training config + hyperparameter rationale

Primary (Session A) mirrors the **public Arushhh QLoRA recipe**; every deviation is called out. Values from `configs/base.yaml` :

Hyperparameter	Value	Rationale (why this, why not something else)
Quantization NF4 double-quant, <code>compute_dtype</code> bf16	<code>load_in_4bit: true, quant_type: nf4</code>	8B fits T4/P100; NF4 > FP4 for instruction-tuned weights (Arushhh recipe)
LoRA r / alpha / dropout	32 / 64 / 0.05	Arushhh recipe verbatim: r=32 captures translation style shift; alpha=2r keeps scaling stable; 0.05 dropout regularizes 8k-sample run
<code>target_modules: all-linear + exclude_modules</code> (vision/audio, <code>lm_head</code>)	see <code>base.yaml</code>	Gemma-4 PEFT uses <code>ClippableLinear</code> ; a bare <code>q_proj</code> list crashes — all-linear+exclude is mandatory (gotcha §6)
<code>modules_to_save: [lm_head, embed_tokens]</code>	—	Lets output embeddings track Marathi distribution without full finetune
LR 1e-4, <code>cosine_with_min_lr</code> (<code>min_lr_rate</code> 0.1), warmup 0.03	—	Arushhh recipe: 1e-4 is the QLoRA sweet spot (1e-5 underfits, 3e-4 diverges on 8k); min-LR floor avoids late collapse; 3% warmup covers scheduler init
<code>max_seq_length</code> 1024, packing off, <code>assistant_only_loss</code>	—	Education pairs fit 1024; packing off keeps src/tgt alignment auditable; loss on assistant span only
Batch 1 × accum 16 (eff. 16), <code>paged_adamw_8bit</code> , wd 0.01, clip 1.0	—	Single-GPU fit with stable effective batch; paged optimizer survives T4 spikes
<code>max_steps</code> 1000, save/eval every 100, logging 10	—	All checkpoints kept (Drive-synced); fits 12h session with ≤100-step granularity
bf16 (+ P100 fallback note)	<code>bf16: true</code>	T4 bf16 OK; P100 lacks bf16/FA2 — use fp16 path (see <code>docs/SETUP.md</code>)
Gradient checkpointing on, <code>remove_unused_columns=False</code>	—	VRAM fit + required so multimodal columns survive collation
Seed 42	—	Deterministic splits + shuffles

Fallback (Session B) deltas: `SEQ_2_SEQ_LM` LoRA on `q_proj,k_proj` only, `attn_implementation="eager"`, `num_workers=0` (Kaggle stability), same 100-step cadence.

6. Gemma-4 gotchas hit

1. `transformers>=5.5.2` **required** for KV-sharing + `mm_token_type_ids`. Older versions

silently misroute multimodal token types. Pinned `transformers==5.13.1` in `requirements.txt`; Session B uses a **separate env** (`transformers>=4.33.2,<5`) because IndicTransToolkit conflicts with v5.

1. PEFT `ClippableLinear` **needs all-linear + exclude — never a bare `q_proj` list.**

Bare lists crash adapter injection on Gemma-4. `configs/base.yaml` encodes the working form.

1. **No DeepSpeed ZeRO-3 for LoRA.** ZeRO-3 shards the frozen base out from under PEFT;

single-GPU QLoRA needs none of it.

1. **vLLM has no runtime LoRA — merge first.** Serve via `inference.py` on merged weights,

not a vLLM LoRA slot.

1. **"Loss drops but inference unchanged" → verify greedy + nonce overfit.** Before trusting

a falling curve: greedy-decode a train sample, then overfit a nonce pair (e.g. a made-up proper noun) for a few steps — if the nonce doesn't stick, the loss is masking/output-shift, not learning. (Procedure in `docs/TRAINING.md`.)

7. Evaluation protocol

- Benchmarks: `in22_gen` + `flores_devtest` per `configs/base.yaml` (`eval.benchmarks`).
- Decoding: `max_new_tokens` 256, `num_beams` 5, batch 8 via `mr_mt/evaluate.py` .
- Metrics: sacreBLEU BLEU + chrF (`score()`), written to `metrics.json` with

`<split>_preds.txt` / `.refs.txt` under `reports/predictions/` .

- Normalize-before-score; test sets never enter `data/processed/train.jsonl`

(gates L1..L10 in `reports/LEAKAGE_CHECKLIST.md`).

TODO(results): paste `metrics.json` summary (BLEU/chrF per benchmark × sessions A/B/C). TODO(results): add `reports/figures/metric_bars.png` + loss curves. TODO(results): qualitative table — 5 wins / 5 failures with src/ref/hyp.

8. Challenges + fixes

#	Challenge	Fix
1	8B doesn't fit Kaggle VRAM full-finetune	4-bit NF4 QLoRA, batch 1×16, grad checkpointing, seq 1024
2	Gemma-4 PEFT crash on targeted <code>q_proj</code> lists	all-linear + exclude list (recipe-mandated)
3	<code>transformers</code> version split (A needs ≥5.5.2, B needs <5)	Two envs; <code>requirements.txt</code> pins A/C, comments pin B
4	12h session timeouts	All checkpoints kept + Drive sync (live rclone / Hub optional); resume-from-checkpoint flow (<code>docs/TRAINING.md</code>)
5	P100 lacks bf16/FA2	fp16 fallback path; T4 preferred for Session A
6	Leakage risk (Samanantar IS BPCC-adjacent pretraining data)	Samanantar pivot (COILD denied) + L1..L10 checklist + normalize-before-score; Session B memorization caveat stated in §4
7	No hosted tracking	TensorBoard per session + <code>reports/experiments.csv</code> index + Drive sync

9. Reproduce

```

pip install -r requirements.txt # Session A/C env (see docs/SETUP.md for Session B env)
make data                      # Samanantar mr -> data/processed/{train,dev,test}.jsonl
make train-a                   # primary Bodhan 8B QLoRA (Session A)
make eval                      # IN22-Gen + FLORES -> reports/metrics.json
make figures                   # loss / length-hist / metric bars -> reports/figures/

```

Resume + multi-account + Drive pull flow: docs/TRAINING.md. Environment + gated access: docs/SETUP.md.

10. What I'd do with more time

1. **Ablation matrix (Session C extension):** $r \in \{16, 32, 64\} \times LR \in \{5e-5, 1e-4\}$ on a fixed

2k subset; log to `reports/experiments.csv` — currently only single-point recipe mirror.

1. **Back-translation augmentation:** Marathi monolingual $\rightarrow$ English pseudo-pairs, filtered by round-trip chrF, to grow general-domain coverage without new parallel data.

1. **chrF++ + COMET:** add learned metric alongside BLEU/chrF for morphologically rich Marathi.

2. **Merged-weight serving:** merge LoRA $\rightarrow$ full checkpoint, quantize (GGUF/AWQ), latency/quality

report on a fixed 500-sentence general-domain slice.

1. **Error taxonomy:** annotate 200 failures (agreement, postpositions, technical terms) to drive the next data slice choice.

README

Overview, structure and how to reproduce — source: README.md

Marathi MT via Bodhan Fine-Tune (English $\rightarrow$ Marathi)

AI4Bharat / AI Research Engineer take-home: pick **one** Bodhan model, fine-tune it for **Marathi**, and deliver an end-to-end run with clean code, honest docs, and reproducible artifacts.

- **Primary:** [bodhan-ai/indic-translate](#)

(Gemma-4 E4B, 8B, gated) fine-tuned with QLoRA.

- **Session B:** [ai4bharat/indictrans2-en-indic-dist-200M](#)

(gated `auto` ; accept once) + IndicTransToolkit, direction `eng_Latn  $\rightarrow$  mar_Deva` .

- **Train:** [ai4bharat/samanantar](#)

config `mr` (OPEN, English $\rightarrow$ Marathi, CC-BY-NC-4.0).

- **Eval (held out, never trained on):** [ai4bharat/IN22-Gen](#)

(config `null/ default` , split test) + [facebook/flores](#) (config `eng_Latn-mar_Deva` , split devtest), direction `eng_Latn  $\rightarrow$  mar_Deva` .

Dataset pivot: the preferred not-in-training education-domain set (`coild-aikosh/Education_v2`) is manual-gated and access was denied, so training pivoted to the OPEN `samanantar` (`mr`) set. Note

Samanantar (via BPCC) WAS used in IndicTrans2 pretraining, so Session B scores partly re-measure memorization.

Scoring note (per assignment brief): NOT scored on metrics; scored on end-to-end run, code structure, and effort/judgement/problem-solving. The graded narrative lives in APPROACH.md.

Repo tree

```
marathi-mt-bodhan/
├── README.md           # this file
├── APPROACH.md         # graded narrative (choices, config rationale, gotchas)
├── Makefile            # data / train-a,b,c / eval / figures / sync / clean
├── .env.example        # env keys WITHOUT secrets (copy to .env, gitignored)
├── configs/
│   └── base.yaml       # shared config; session configs inherit + override
├── src/mr_mt/
│   ├── config.py       # package root (import as `mr_mt`, PYTHONPATH=src)
│   ├── load_config / load_session_config (deep merge)
│   ├── secrets.py      # get_hf_token (Kaggle Secret/Dataset/env/.env)
│   ├── utils.py        # seeds, JSONL IO, experiments.csv append
│   ├── checkpointing.py # CheckpointMirrorCallback (background rclone mirror + pruning)
│   ├── compat.py       # supported_kwargs: keyword/version compat for the two transformers s
│   ├── data/           # download.py, prepare.py, decontaminate.py
│   ├── train_bodhan_qlora.py # Session A (primary, 8B QLoRA)
│   ├── train_indictrans2_lora.py # Session B (en-indic-dist-200M LoRA)
│   ├── evaluate.py     # IN22-Gen + FLORES scoring -> metrics.json
│   ├── inference.py    # single-string translate CLI
│   ├── demo.py         # fixed-sentence demo + optional hosted-API comparison
│   └── plots.py        # loss / length-hist / metric-bar figures
├── scripts/
│   ├── kaggle/         # Kaggle SCRIPT kernels (no notebooks): kaggle_env + kernel_*.py + me
│   ├── run_train_sessionA.sh, run_train_sessionB.sh, run_data.sh, run_eval.sh
│   ├── pull_kaggle_output.ps1 # Kaggle Output -> local artifacts/
│   └── sync_drive.ps1   # artifacts/ -> gdrive:mr-mt-edu-2026/
├── docs/
│   ├── SPEC.md         # interface spec (function contracts, config schema)
│   ├── SETUP.md        # HF login, Kaggle Dataset token, two envs, GPU notes
│   ├── HYPERPARAMETERS.md # every knob: why this value, when/how to shift
│   └── TRAINING.md     # per-session runs, resume, monitoring, pull flow
├── data/
│   ├── raw/            # gitignored (except .gitkeep)
│   └── processed/      # train/dev/test.jsonl, gitignored (except .gitkeep)
├── reports/
│   ├── LEAKAGE_CHECKLIST.md # L1..L10 decontamination checklist
│   ├── experiments.csv   # run log (TensorBoard is primary; this is the index)
│   ├── metrics.json     # eval output (TODO until runs complete)
│   ├── figures/         # plots.py output
│   └── predictions/     # <split>_preds.txt + .refs.txt
├── models/             # gitignored weights staging (adapters only in Hub/Drive)
├── vendor/             # NO-LICENSE local references only, gitignored, never committed
└── requirements.txt     # Session A/C stack pinned; Session B installed separately
```

Quickstart

```
pip install -r requirements.txt
make data
make train-a
```

- `make data` builds `data/processed/{train,dev,test}.jsonl` from `configs/base.yaml` .
- `make train-a` runs the primary Bodhan 8B QLoRA fine-tune (Session A).
- `make demo` prints Marathi translations for the built-in English sentence set

(adds a Bodhan hosted-API comparison when `BODHAN_API_KEY` / `BODHAN_API_URL` are set).

- See [\[docs/SETUP.md\]\(docs/SETUP.md\)](#) before anything gated, and

[\[docs/TRAINING.md\]\(docs/TRAINING.md\)](#) for sessions B/C, resume, and monitoring.

Config (from `configs/base.yaml`)

Key	Value	Why (one line)
<code>model.name</code>	<code>bodhan-ai/indic-translate</code>	Assignment pick: one Bodhan model for Marathi
<code>lora.r</code> / <code>alpha</code> / <code>dropout</code>	32 / 64 / 0.05	Mirror public Arushhh recipe (see APPROACH.md §5)
<code>lora.target_modules</code>	<code>all-linear</code> + <code>exclude_modules</code> (vision/audio, <code>lm_head</code>)	Required for Gemma-4 PEFT <code>ClippableLinear</code>
<code>training.learning_rate</code>	1e-4, <code>cosine_with_min_lr</code> (<code>min_lr_rate</code> 0.1)	Arushhh recipe; floor avoids late-training stall
<code>training.max_seq_length</code>	1024	Education sentences fit; keeps 8B QLoRA on T4/P100
<code>training batch</code>	per-device 1 × accum 16	Effective batch 16 on a single Kaggle GPU
<code>training.save_steps</code> / <code>eval_steps</code>	100 / 100	Resume-friendly cadence for 12h Kaggle sessions
<code>data.direction</code>	<code>eng_Latn->mar_Deva</code>	English → Marathi
<code>prepare.max_train</code> / <code>max_dev</code>	8000 / 1000	Bounded run for 30h/week Kaggle budget
<code>eval decoding</code>	256 new tokens, beam 5	IN22-Gen/FLORES scoring config
<code>report_to</code>	<code>tensorboard</code>	Deliberate: TB + CSV, no MLflow server / W&B (see APPROACH.md §2)

Session configs (sessions A/B/C) inherit `base.yaml` via `mr_mt.config.load_session_config` and override only `run.name` , `output_dir` , `model/lora` blocks, and `hub.repo_id` .

Checkpoints (all of them, in full)

Training keeps **all** checkpoints (`save_total_limit`: `null`) with full optimizer state (`save_only_model`: `false`), and gets them off the ephemeral Kaggle VM:

1. **Live Drive mirror (active, background):** each account has a private `gdrive-creds`

Dataset (rclone.conf); kaggle_env installs rclone and points RCLONE_CONFIG at it, and CheckpointMirrorCallback rclones each checkpoint to gdrive:mr-mt-edu-2026/<run>/checkpoints in the background as it is saved (training never stalls; full checkpoints by default; adapter_only_copy: true for small adapter-only copies). Each verified upload gets a _upload_complete marker; local copies beyond the newest checkpointing.keep_local are pruned once uploaded, so /kaggle/working stays under quota. Resume after a 12h kill: set the Kaggle Secret MR_MT_RESUME=auto and re-push the kernel.

1. **Post-run Drive sync (fallback):** scripts/pull_kaggle_output.ps1 →

scripts/sync_drive.ps1 copies the whole run tree to gdrive:mr-mt-edu-2026/<run>/.

1. **HF Hub (optional, off by default):** set hub.push_to_hub: true + a real

hub.repo_id + a write-role token; hub.strategy: all_checkpoints then streams every checkpoint for cross-session resume.

Running on Kaggle (no notebooks)

Kaggle kernel_type: script entrypoints live in scripts/kaggle/. kaggle kernels push -p <dir> requires a literal kernel-metadata.json, so push one kernel at a time with the helper (it stages the metadata and uses that task's account token):

```
powershell -ExecutionPolicy Bypass -File scripts/kaggle/push_kernel.ps1 -Task bodhan # acct1 (p
powershell -ExecutionPolicy Bypass -File scripts/kaggle/push_kernel.ps1 -Task indictrans2 # acct2 (f
```

The token arrives via the attached private hf-token Dataset (or the HF_TOKEN Kaggle Secret) and the repo URL is set in kaggle_env.py; train kernels auto-install the pinned stack and auto-prepare data/processed when missing.

See scripts/kaggle/README.md for the full flow.

Model & dataset links

Artifact	Link	License / access
Primary model <code>bodhan-ai/indic-translate</code>	https://huggingface.co/bodhan-ai/indic-translate	Gated; Indic Open Model License v1.0 (share-alike)
Base weights <code>google/gemma-4-E4B-it</code>	https://huggingface.co/google/gemma-4-E4B-it	Gated (accept license)
Session B <code>ai4bharat/indictrans2-en-indic-dist-200M</code>	https://huggingface.co/ai4bharat/indictrans2-en-indic-dist-200M	Gated <code>auto</code> (one-time accept)
Train <code>ai4bharat/samanantar</code> (config <code>mr</code> , <code>en→mr</code>)	https://huggingface.co/datasets/ai4bharat/samanantar	OPEN; CC-BY-NC-4.0
Eval <code>ai4bharat/IN22-Gen</code> (config <code>null</code> , split <code>test</code>)	https://huggingface.co/datasets/ai4bharat/IN22-Gen	Accept license
Eval <code>facebook/flores</code> (config <code>eng_Latn-mar_Deva</code> , split <code>devtest</code>)	https://huggingface.co/datasets/facebook/flores	Already accessible (accept conditions if prompted)
Session artifacts (Drive)	<code>gdrive:mr-mt-edu-2026/{bodhan-qlora,indictrans2-lora,eval}</code> — share URL: <!-- TODO: paste Drive link after runs -->	All checkpoints + <code>metrics.json</code>

License / attribution

- Code in this repo: <!-- TODO: confirm repo license (e.g. MIT) -->.
- `bodhan-ai/indic-translate` is under the **Indic Open Model License v1.0** (share-alike):

derivatives must carry the same license terms and the attribution **"Built with indic-translate from Bodhan AI / AI4Bharat"**.

- `vendor/` holds no-license third-party references for local reading only —

gitignored, never committed (see `.gitignore`).

Further reading

- `APPROACH.md` — the full graded narrative + results TODOs.
- `docs/SETUP.md` — environment, gated access, Kaggle secrets.
- `docs/HYPERPARAMETERS.md` — hyperparameter rationale + shift table.
- `docs/TRAINING.md` — running/resuming sessions, Drive pull flow.
- `docs/SPEC.md` — module/function contracts for contributors.
- `reports/LEAKAGE_CHECKLIST.md` — decontamination gates.

HYPERPARAMETERS

Every hyperparameter and why — source: `docs/HYPERPARAMETERS.md`

HYPERPARAMETERS — what we run, why, and when to change it

Single source of truth: `configs/` . `base.yaml` holds every training/eval knob; `session{A,B,C}*.yaml` override only what differs. Python code contains only safe fallbacks (`dict.get` defaults) — if a value matters, it lives in YAML. To change anything below, edit the YAML, never the code.

Session A — Bodhan Gemma-4 (8B) QLoRA (primary)

Provenance: the combination is mirrored from a recipe already proven on **this exact model + dependency stack** (transformers 5.13.1 / trl 1.6.0 / peft 0.20.0 on a Kaggle T4; 8/10 runs converged, COMET $\approx$ 74.5 reference). We change only what the assignment forces (Marathi target, Samanantar en→mr data — the preferred education-domain set, `coild-aikosh/Education_v2` , was manual-gated and access was denied).

Knob (config path)	Value	Why this value	When/how to shift
<code>model.load_in_4bit + bnb.quant_type: nf4 + double_quant: true</code>	on	8B on a 16GB T4 is impossible in fp16 (16GB weights alone). NF4+double-quant is the standard QLoRA recipe (Dettmers et al., 2023) — ~5GB weights leaves room for activations+optimizer.	No alternative on T4.
<code>bnb.compute_dtype: bfloat16</code>	bf16	bf16 has fp32-range exponent → no overflow NaNs (unlike fp16).	T4 (sm_75) has no native bf16: on NaN/bnb dtype errors flip the documented fp16 path (<code>training.bf16:false</code> , <code>fp16:true</code> , <code>compute_dtype:float16</code>) — see SETUP.md §4.
<code>lora.r: 32</code> , <code>lora.alpha: 64</code>	32/64	$\alpha/r = 2$ keeps the proven effective adapter LR; $r=32$ is the sweet spot for domain-adapting an 8B on ~8k pairs ($r=16$ underfits slightly, $r=64$ gains nothing at this data size and costs VRAM). Proven values.	Session C ablates $r=16/\alpha=32$. Raise r only if dev loss plateaus far above train loss AND more data is available.
<code>lora.target_modules: all-linear + exclude_modules</code> (vision/audio)	all text linears	Attention-only LoRA underperforms all-linear for MT style transfer; vision/audio towers are excluded (text-only task) — saves memory, protects encoders.	Fixed.
<code>lora.modules_to_save: [lm_head]</code>	lm_head	Devanagari target distribution shift; training the head lets Marathi token logits adapt. (<code>embed_tokens</code> is listed in <code>base.yaml</code> but deliberately stripped in code — Gemma ties input/output embeddings.)	Fixed.
<code>lora.dropout: 0.05</code>	0.05	Light regularization for 8k rows.	Raise to 0.1 if dev loss diverges from train.
<code>training.learning_rate: 1e-4</code>	1e-4	Proven on this stack; QLoRA typical band is $1e-4$ – $2e-4$.	NaN/spikes → $5e-5$. Plateaued high loss → $2e-4$.
<code>lr_scheduler_type: cosine_with_min_lr + min_lr_rate: 0.1</code>	cosine, 10% floor	Smooth decay for convergence without killing tail-end learning; proven.	Fixed.
<code>warmup_ratio: 0.03</code>	3% (30 steps)	Standard short-run warmup; stabilizes 8-bit Adam without burning budget. transformers 5.x folded this into <code>warmup_steps</code> (float in <code>[0, 1]</code> = ratio) and deprecates <code>warmup_ratio</code> , so Session A forwards the ratio through <code>warmup_steps</code> (<code>_warmup_value</code>).	Fixed.
<code>max_seq_length: 1024</code>	1024	Rows are ≤ 100 words (prepare filters), so 1024 covers p99 + template overhead; attention cost grows steeply with length on T4.	OOM → 768 (near-zero truncation at the 100-word cap).

Knob (config path)	Value	Why this value	When/how to shift
<code>per_device_train_batch_size:</code> <code>1 ×</code> <code>gradient_accumulation_steps:</code> <code>16</code>	eff. 16	VRAM-bound at 8B/4-bit/seq-1024; eff 16 gives stable gradients.	VRAM headroom >2GB → bs=4/accum=4 (same eff 16), ~10-20% faster steps. Only after smoke test.
<code>max_steps: 1000</code> (= 2 epochs of 8k)	1000	1000 × 16 = 16k samples = 2 epochs — proven; a 3rd epoch overfits 8k rows.	Deadline pressure → 600-800 (checkpoints every 100, stop anywhere).
<code>save_steps/eval_steps: 100</code> , <code>logging_steps: 10</code>	100/100/10	A 12h Kaggle kill must cost ≤ 1 save interval; eval-at-save gives the selection curve free.	Do NOT raise save_steps (TRAINING.md).
<code>optim: paged_adamw_8bit</code>	paged 8-bit	QLoRA-standard: pages states to CPU on VRAM spikes, 8-bit halves optimizer memory.	Fixed.
<code>gradient_checkpointing: true</code> , <code>use_cache: false</code>	on	Required to fit; ~30% step slowdown is the price.	Fixed on T4.
<code>packing: false</code>	off	TRL 1.6 packing is incompatible with <code>assistant_only_loss</code> ; losing assistant masking hurts more than packing helps at seq 1024 with short rows.	Fixed.
<code>assistant_only_loss: true</code>	on	Loss on Marathi answer tokens only — the prompt is an instruction; training on it wastes capacity and teaches prompt-echoing. Needs <code>{% generation %}</code> template markers (fallback documented in <code>_load_chat_dataset</code>).	Fixed.
<code>weight_decay: 0.01</code> , <code>max_grad_norm: 1.0</code> , <code>seed: 42</code>	defaults	HF-standard, proven; shared seed for cross-session comparability.	Fixed.

Session B — IndicTrans2 en-indic-dist-200M LoRA

Knob	Value	Why
<code>model.name</code>	indictrans2-en-indic-dist-200M	En-Indic distilled checkpoint (200M): native English source → Marathi, matching the <code>eng_Latn->mar_Deva</code> Samanantar train direction. Gated <code>auto</code> — one-time accept.
<code>lora.r:16/alpha:32/dropout:0.1 , targets q_proj,k_proj</code>	small	200M seq2seq needs far less adapter capacity than 8B; q/k-only mirrors known-good IndicTrans2 LoRA fine-tunes; heavier dropout suits the small model.
<code>learning_rate: 2e-4 , inverse_sqrt , warmup_steps: 1000</code>	—	inverse_sqrt+warmup is the classic Noam-style NMT schedule; 2e-4 suits LoRA on a 200M. Session B config sets <code>warmup_steps: 1000</code> and <code>warmup_ratio: 0</code> (explicit step warmup, not the base 0.03 ratio).
<code>max_seq_length: 256</code>	256	Sentence-level MT; IndicTrans2 convention.
<code>per_device 8 × accum 16 (eff 128)</code>	128	Standard NMT token-batch equivalent for stable seq2seq gradients.
<code>max_steps: 3000 , save/eval 500</code>	—	$3000 \times 128 \approx 48$ passes over 8k — small model, needs many epochs; small checkpoints, so 500-step cadence is cheap.
<code>fp16: true , eager attention, dataloader_num_workers: 0</code>	—	Documented stability guards for the IndicTrans2 random-segfault issue (upstream #117); do not "optimize" these away.
<code>predict_with_generate: false</code>	off	Beam-5 generation every 500 steps over ~1000 dev rows wastes 20-50 min/run; loss-only eval suffices for model selection. Real BLEU/chrF++ come from <code>mr_mt.evaluate</code> .

Eval (both families) — `eval:` in `base.yaml`

Knob	Value	Why
<code>num_beams: 5</code>	5	MT-benchmark standard; comparable to published IN22/FLORES numbers.
<code>max_new_tokens: 256</code>	256	Matches the 100-word row cap with margin.
<code>batch_size: 8</code>	8	T4 4-bit decode sweet spot; sources are length-sorted before batching (less padding, ~15-30% faster).
metrics	sacrebleu BLEU + chrF++ (<code>word_order=2</code>)	chrF++ is robust for morphologically rich Marathi; BLEU for comparability; signatures printed for reproducibility.

Quick shift table (if the smoke test / run says X)

Symptom	Change (YAML only)
OOM at step 1	<code>max_seq_length: 768</code> (bs stays 1)
NaN/inf loss on T4	fp16 path per SETUP.md §4; if persists, <code>learning_rate: 5e-5</code>
Train loss flat, dev flat-high	<code>learning_rate: 2e-4</code> OR <code>max_steps: 1500</code> — pick one
Dev loss rising while train falls	<code>lora.dropout: 0.1</code> ; pick an earlier checkpoint (saved every 100)
Not enough time	<code>max_steps: 600</code> , evaluate checkpoint-600 — do not touch LR
VRAM headroom >2GB	<code>per_device_train_batch_size: 4</code> , <code>gradient_accumulation_steps: 4</code> (same eff. 16) for ~10-20% faster steps

SPEC

Interface spec — source: docs/SPEC.md

INTERFACE SPEC — marathi-mt-bodhan

All code targets Python 3.11 (Kaggle). Package root: `src/mr_mt` imported as `mr_mt` (add `src` to PYTHONPATH; configs reference `src` not used at runtime directly).

Conventions

- Config = YAML loaded by `mr_mt.config.load_config(path) -> dict` .
- Dataset rows (JSONL): `{"src": str, "tgt": str, "src_lang": str, "tgt_lang": str, "domain": str}` .

`src_lang` / `tgt_lang` use FLORES codes (e.g. `hin_Deva` , `mar_Deva` , `eng_Latn`).

- Processed files: `data/processed/train.jsonl` , `dev.jsonl` , `test.jsonl` .
- Entry points run as modules: `python -m mr_mt.<script> --config <yaml> [args]` .
- Secrets: read `HF_TOKEN` from env first, else from a gitignored `.env` at repo root.

Never hardcode tokens. Use `mr_mt.secrets.get_hf_token()` .

- Outputs: training writes to `output_dir` from config; adapters under `<output_dir>/adapter` .
- All scripts must be importable without side effects (`if __name__ == "__main__":`).

Modules and required public functions

mr_mt/secrets.py

- `get_hf_token()` -> Optional[str] # env -> Kaggle Secret -> hf-token Dataset -> .env
- `get_env_secret(name)` -> Optional[str] # env -> .env, for non-HF knobs (e.g. BODHAN_API_KEY)
- `iter_hf_tokens()` -> Iterator[Tuple[str, str]] # (source label, token) candidates, deduped

- `hf_token_can_access(token, repo_type, repo_id) -> Optional[bool] # True/False(401|403)/None(undecidable)`
- `select_hf_token(required=(), timeout=20) -> Tuple[Optional[str], str] # first candidate that`

can read every `(repo_type, repo_id)` in `required`; falls back to the first with a warning

mr_mt/utils.py

- `set_seed(seed: int) -> None`
- `ensure_dir(p) -> str`
- `read_jsonl(path) -> List[dict]`
- `write_jsonl(rows, path) -> None`
- `log_experiment(row: dict, path: str) -> None # append CSV, create header if missing`
- `EXPERIMENT_COLUMNS: List[str]`

mr_mt/config.py

- `load_config(path) -> dict`
- `resolve(base_cfg: dict, overrides: Optional[dict]) -> dict # deep merge`

mr_mt/data/download.py

- `download_dataset(cfg: dict) -> str # snapshot_download / load_dataset; returns local dir`
- `download_benchmarks(cfg: dict) -> Dict[str, str] # eval sets`

mr_mt/data/prepare.py

- `build_splits(cfg: dict) -> dict # reads raw dir, writes train/dev/test jsonl; returns counts`
- CLI `main()` with `--config`.

mr_mt/data/decontaminate.py

- `normalize(text, lang) -> str`
- `pair_hash(src, tgt) -> str`
- `dedup_against(rows, blacklist_hashes) -> Tuple[List[dict], int]`
- `near_dup_filter(rows, refs, threshold) -> Tuple[List[dict], int]`

mr_mt/train_bodhan_qlora.py (Session A primary)

- `build_prompt(sample, cfg) -> str # chat-template-ready text`
- `load_model_and_tokenizer(cfg) -> Tuple[model, tokenizer, processor]`
- `build_lora(cfg) -> peft.LoraConfig`
- `main() -> SFTTrainer, save adapter to <output_dir>/adapter, optional hub push.`
- Must: 4-bit NF4 via BitsAndBytesConfig; `target_modules="all-linear"` with

`exclude_modules`; `max_length=max_seq_length`; `remove_unused_columns=False` for mm; `gradient_checkpointing` when configured; `neftune_noise_alpha` optional.

mr_mt/train_indictrans2_lora.py (Session B fallback)

- `load_processor_and_model(cfg)` using `IndicTransToolkit.IndicProcessor` +

`AutoModelForSeq2SeqLM(trust_remote_code=True)` .

- `build_lora(cfg)` -> `LoraConfig(task_type=SEQ_2_SEQ_LM, target q_proj,k_proj)`.
- `main()` -> `Seq2SeqTrainer`; `num_workers=0`; `attn_implementation="eager"`; adapter to `<out>/adapter` .

mr_mt/evaluate.py

- `translate_batch(model, tokenizer/processor, samples, cfg)` -> `List[str]`
- `score(preds, refs, tgt_lang)` -> `dict # {"bleu":...,"chr":...}, sacrebleu`
- `main()` -> writes `metrics.json` and `<split>_preds.txt + .refs.txt` under `reports/predictions`.
- Benchmarks configured as list of `{dataset, config, split, src_lang, tgt_lang}`.

mr_mt/inference.py

- `translate(text, model, tokenizer, cfg, family="bodhan")` -> `str`
- CLI `--config --adapter --text --family` .

mr_mt/demo.py

- `DEMO_SENTENCES: List[str]` — fixed Hindi demo set.
- `hosted_translate(sentence, api_url, api_key, ...)` -> `Optional[str]` — best-effort

Bodhan hosted-API call (returns `None` on failure; never raises).

- CLI `--config --adapter --family [--sentences ...]` — prints HI/MR pairs and, when

`BODHAN_API_KEY / BODHAN_API_URL` resolve (env → `.env`), the hosted output too.

mr_mt/plots.py

- `plot_loss(trainer_state_or_csv, out_png)`
- `plot_length_hist(rows, out_png)`
- `plot_metric_bars(metrics: dict, out_png)`

mr_mt/compat.py

- `supported_kwargs(target, kwargs, aliases=None)` -> `Tuple[dict, List[str]]` —

renames/filters kwargs against the *installed* signature of `target` (class or callable) and returns (kept, dropped) . Callers log the dropped names loudly.

- `ALIASES` maps renamed keywords: `evaluation_strategy` -> `eval_strategy`

(`transformers >= 4.46`), `tokenizer` -> `processing_class`, and `generation_max_length` -> `generation_max_new_tokens` for forward compatibility.

- Rationale: Session B's pin (`>=4.33.2,<5`) spans the rename point, where the old

names were **removed, not aliased**; Session A pins 5.13.1 where `warmup_ratio` is deprecated in favour of `warmup_steps` (float in `[0, 1)` = ratio).

mr_mt/checkpointing.py

- ``CheckpointMirrorCallback(mirror_dir, adapter_only_copy, rclone_remote,`

`rclone_binary, keep_local, max_pending)` – `TrainerCallback`. `on_save` : with `adapter_only_copy: true` it writes an adapter-only copy of each `checkpoint-<step>` under `mirror_dir` (NOT resumable) and `background-rclones` it; with `false` (default) it `background-rclones` the full checkpoint dir directly. Uploads are `subprocess.Popen` (never block training), reaped on the next save, all awaited at `on_train_end`; a `_upload_complete` marker is touched per verified upload; confirmed checkpoints beyond the newest `keep_local`` are pruned locally. Best-effort: failures never abort training, but a configured remote with a missing `rclone` binary prints a loud warning.

- `build_mirror_callback(cfg) -> CheckpointMirrorCallback` from `cfg["checkpointing"]`.

scripts/kaggle/ (Kaggle script kernels, not notebooks)

- `kaggle_env.activate(stack) -> repo_dir` — locate/clone repo, add `src` +

`scripts/kaggle` to path, `chdir`, export HF token, optional pip install (`MR_MT_INSTALL=1`) and `rclone` config.

- `kaggle_env.get_setting(name, default="")` — env var, then Kaggle Secret.
- `kaggle_env.verify_hf_access(stack) -> {"token", "source", "results"}` — probes the

stack's gated repos and returns per-repo `True / False (401|403) / None`; `activate()` runs it **before** the pip install, prints an `ACTION REQUIRED` block, and raises `SystemExit` (fail-fast) when a repo is definitively blocked or no token candidate exists. `None` (unknown) never aborts; `MR_MT_TOKEN_PROBE=0` skips probe + abort.

- `python scripts/kaggle/kaggle_env.py --check-access <stack>` — the same probe as a

local pre-flight CLI (exit 1 when a repo is definitively blocked).

- `kaggle_env.latest_confirmed_checkpoint(remote) -> Optional[str]` — highest

`checkpoint-<N>` under a Drive remote that has the `_upload_complete` marker.

- `kaggle_env.rclone_fetch(remote_dir, local_dir, includes=None) -> bool`.
- `kernel_prepare_data.py`, `kernel_train_bodhan.py`, `kernel_train_ablation.py`,

`kernel_train_indictrans2.py`, `kernel_evaluate.py`, `kernel_probe_token.py` — thin entrypoints; the train kernels auto-run download+prepare if `data/processed` is missing and honour `MR_MT_RESUME=auto`; every `checkpoint-<step>` is auto-uploaded to Drive in the background with a `_upload_complete` marker; the Bodhan train kernels (Session A/C) run in-kernel post-train eval on the just-saved local adapter (never fails training); the eval kernel auto-fetches adapter files only from the latest confirmed Drive checkpoint when `MR_MT_ADAPTER` is unset; the probe kernel (tasks `probe / probe2 / probe3`, CPU-only) lists every HF token candidate an account can see with `whoami` + gated-repo verdicts. `kernel-metadata.*.json` set `kernel_type: script`, GPU, internet, and the private `hf-token / gdrive-creds` datasets.

Config schema (configs/base.yaml)

See `configs/base.yaml`. Session configs inherit and override.

SETUP

Environment setup — source: docs/SETUP.md

SETUP

1. Hugging Face login + gated licenses to accept

All tokens via `HF_TOKEN` (env first, else gitignored `.env` at repo root — `mr_mt.secrets.get_hf_token()`). **Never commit tokens**; `.env` is gitignored.

1. Create a token at <https://huggingface.co/settings/tokens> (read role is enough to download; write role only if pushing checkpoints to the Hub).

1. One-time gated acceptances (while logged in with the `HF_TOKEN` account):
2. `ai4bharat/samanantar` (train, config `mr`) — OPEN, no acceptance needed.
3. `ai4bharat/IN22-Gen` — <https://huggingface.co/datasets/ai4bharat/IN22-Gen> (accept license)
4. `facebook/flores` — <https://huggingface.co/datasets/facebook/flores> (already accessible; accept conditions if prompted)
5. `bodhan-ai/indic-translate` — <https://huggingface.co/bodhan-ai/indic-translate>
6. `google/gemma-4-E4B-it` — <https://huggingface.co/google/gemma-4-E4B-it>
7. `ai4bharat/indictrans2-en-indic-dist-200M` — <https://huggingface.co/ai4bharat/indictrans2-en-indic-dist-200M> (gated `auto`; Session B — accept its conditions once)
8. Rejected: `coild-aikosh/Education_v2` was the preferred not-in-training

education set, but it is manual-gated and access was denied — NOT used.

1. Local login:

```
copy .env.example .env
:: edit .env, set HF_TOKEN=<your_token_here>
```

Or export `HF_TOKEN` in the environment (takes precedence over `.env`).

2. Kaggle token + code delivery (script kernels)

We run Kaggle **script kernels** (`kernel_type: script`), not notebooks. Kaggle Secrets are **user-level** — created once per account (web UI → Settings/Add-ons → Secrets) they are readable from every kernel of that account, including pushed script kernels, via `UserSecretsClient`:

1. **Preferred:** create a Secret named `HF_TOKEN` on each of the three

accounts. (kernel-metadata.json has no secrets field and the CLI has no secrets flag — `kaggle-api#582` is an *enhancement* request for CLI-side management, NOT a limitation on reading secrets from kernels.)

1. **Fallback (already wired):** a private Kaggle Dataset containing a single

hf_token.txt , attached in each scripts/kaggle/kernel-metadata.*.json under dataset_sources (e.g. kaustubhcrathi/hf-token ; per-account list in scripts/kaggle/README.md).

1. mr_mt.secrets.get_hf_token() resolves the token in this order: HF_TOKEN env →

Kaggle Secret → /kaggle/input/**/hf_token.txt → gitignored .env . It is exported as HF_TOKEN / HUGGING_FACE_HUB_TOKEN in scripts/kaggle/kaggle_env.py . Optional run knobs (MR_MT_RESUME , MR_MT_ADAPTER , MR_MT_FAMILY) likewise read env → Secret (kaggle_env.get_setting).

Code delivery: kaggle_env.py locates the repo at MR_MT_REPO_DIR (default /kaggle/working/marathi-mt-bodhan) and git clone s it from MR_MT_REPO_URL if missing. **Never put .env or the token in the repo.**

Live Drive mirror (wired): each account has a private gdrive-creds Dataset containing rclone.conf ; kaggle_env._configure_rclone() points RCLONE_CONFIG at it and installs the static rclone binary to /kaggle/working/bin if absent. The session configs set checkpointing.rclone_remote , so every checkpoint is rcloned to Drive during training (see TRAINING.md).

Local Drive sync (one-time, per machine). The gdrive: remote lives in your rclone profile (%APPDATA%\rclone\rclone.conf), never in the repo. On a new machine: rclone config → new remote named gdrive → storage drive → client_id/secret blank → scope drive.file → auto config **yes** → browser login, then rclone lsd gdrive: to verify. Staging is the gitignored artifacts/ inside the repo: pull a kernel's output with scripts/pull_kaggle_output.ps1 -Slug <owner/slug> -Acct <1|2|3> (per-account ~/.kaggle/access_token[_acct2|_acct3]), then scripts/sync_drive.ps1 (or make sync) to copy artifacts/ → gdrive:mr-mt-edu-2026/ . The rclone binary path is overridable via the gitignored scripts/rclone_path.txt , falling back to rclone on PATH . The same gdrive-creds credentials are mounted inside Kaggle kernels (_find_rclone_conf).

GPU + persistence

Enable GPU and Internet in kernel-metadata.*.json (enable_gpu , enable_internet). /kaggle/working is ephemeral (12h cap, ~20GB) and only persists on a **successful** run — that is why every checkpoint is mirrored to Drive (mr_mt.checkpointing ; Hub push is an optional second channel). See TRAINING.md.

Pre-flight check (10 s — catches the #1 launch failure). Verify the token can actually read the gated repos a session needs before pushing a 12h kernel:

```
python scripts/kaggle/kaggle_env.py --check-access bodhan # or indictrans2
```

Exit 0 = all probes OK. BLOCKED = a definite 401/403, i.e. either the token is invalid/expired (check <https://huggingface.co/settings/tokens> — whoami must succeed) or the account was never granted access (open the repo page and accept the licence — e.g. ai4bharat/indictrans2-en-indic-dist-200M is a gated auto repo that needs a one-time accept). unknown is inconclusive, not a failure. Kernels run the same probe at startup — before the pip install — print an ACTION REQUIRED block naming the repos that failed, and **abort (SystemExit) immediately** when a repo is definitely blocked or no token candidate exists; set the Secret MR_MT_TOKEN_PROBE=0 to bypass. (Historical note: coild-aikosh/Education_v2 was a MANUAL gate whose approval could take hours — access was denied, so it is no longer probed or used.) To verify a rotation *per Kaggle account* (Secret included — invisible locally), push the CPU-only probe kernel: push_kernel.ps1 -Task probe|probe2|probe3 (acct1/2/3), then read kaggle kernels logs <slug> for whoami=OK as <user> .

Rotating the token means updating `HF_TOKEN` in *both* the local `.env` and the private `hf-token` Dataset (or the `HF_TOKEN` Kaggle Secret) for **every** account: a stale Kaggle Secret outranks the Dataset, though the startup probe now skips a token that fails its gated-repo checks. Confirm with the probe kernel on each account before re-launching a 12h session.

3. Dependencies — TWO environments (do not mix)

Session A / C (Bodhan Gemma-4 QLoRA) — pinned to the Arushhh-proven stack. `transformers>=5.5.2` is mandatory (KV-sharing + `mm_token_type_ids`); pinned `==5.13.1`:

```
pip install -r requirements.txt
```

Pins (see `requirements.txt`): `transformers==5.13.1`, `trl==1.6.0`, `peft==0.20.0`, `datasets==5.0.1`, `tokenizers==0.22.2`, plus `accelerate`, `bitsandbytes`, `sentencepiece`, `sacrebleu`, `huggingface_hub`, `PyYAML`, `pandas`, `matplotlib`, `tensorboard`.

Session B (IndicTrans2 en-indic-dist-200M) — separate env; IndicTransToolkit conflicts with `transformers>=5`:

```
pip install "transformers>=4.33.2,<5" IndicTransToolkit==1.1.1 datasets sacrebleu \
  huggingface_hub PyYAML pandas matplotlib tensorboard peft
:: optional — only if `trust_remote_code` is blocked in your environment, install
:: the AI4Bharat interface instead:
:: git clone --depth 1 https://github.com/AI4Bharat/IndicTrans2 /tmp/IndicTrans2
:: pip install /tmp/IndicTrans2/huggingface_interface
```

Never install both stacks in one env — Session B's `<5` pin will break Session A's Gemma-4 path and vice versa. One Kaggle script kernel per session keeps this clean.

4. GPU notes (T4 vs P100)

GPU	Notes
T4 (preferred for Session A)	T4 (Turing, sm_75) has no native bf16 — bf16 is emulated and bitsandbytes 4-bit bf16 compute can error or NaN. Try bf16 first (<code>training.bf16: true</code> in <code>configs/base.yaml</code>) only if it trains stably; on any NaN/bnb dtype error flip to the fp16 path exactly like the P100 row (<code>training.bf16: false</code> , <code>training.fp16: true</code> , <code>model.bnb.compute_dtype: float16</code>). 8B QLoRA seq 1024 batch 1×16 fits with grad checkpointing
P100	No bf16, no FlashAttention-2. If assigned a P100, flip to the fp16 path (<code>bf16: false</code> , <code>fp16: true</code>) and expect slower steps; keep seq 1024, do not raise batch
Either	12h/session cap — rely on save-every-100 + Hub resume, not on finishing in one go

Verify GPU before launching: `nvidia-smi` (in the run script, or via `kaggle kernels logs`) and confirm dtype flags match the card. Session B (en-indic-dist-200M, 200M params) runs fine on either card.

TRAINING

Per-session runs, resume and monitoring — source: docs/TRAINING.md

TRAINING

Sessions: **A** = Bodhan 8B QLoRA (Acct1, primary) · **B** = IndicTrans2 en-indic-dist-200M LoRA (Acct2) · **C** = ablation/demo (Acct3). Config: `configs/base.yaml` (session configs inherit via `mr_mt.config.load_session_config` and override `run.name`, `output_dir`, `hub.repo_id`). All sessions train `eng_Latn->mar_Deva` on `ai4bharat/samanantar` (config `mr`); eval is IN22-Gen (test) + FLORES (devtest).

Session A — Bodhan 8B QLoRA (primary)

```
make train-a
# = PYTHONPATH=src python -m mr_mt.train_bodhan_qlora --config configs/sessionA_bodhan_qlora.yaml
```

- 4-bit NF4, all-linear LoRA ($r=32/\alpha=64/\text{dropout}=0.05$), lr $1e-4$

`cosine_with_min_lr`, seq 1024. Adapter → `<output_dir>/adapter`.

- Hub push every 100 steps (`hub.strategy: all_checkpoints`) when `push_to_hub: true`

with the session's `hub.repo_id`.

Session B — IndicTrans2 en-indic-dist-200M LoRA

Use the **Session B env** (`transformers<5` + IndicTransToolkit — see `docs/SETUP.md`).

```
make train-b
# = PYTHONPATH=src python -m mr_mt.train_indictrans2_lora --config configs/sessionB_indictrans2_lora
```

- Base model `ai4bharat/indictrans2-en-indic-dist-200M` (gated `auto` — accept

its conditions once with the `HF_TOKEN` account before running).

- `SEQ_2_SEQ_LM` LoRA on `q_proj,k_proj`; `attn_implementation="eager"`;

`num_workers=0`; direction `eng_Latn->mar_Deva`.

Session C — ablation / demo

```
make train-c
# same entry point as A; override r/LR/steps in the session config for the ablation
```

Resume (all sessions)

Resume is **not** automatic: both trainers (`train_bodhan_qlora.py`, `train_indictrans2_lora.py`) only resume when `--resume_from_checkpoint <path>` is passed explicitly (bare `--resume_from_checkpoint` with no value resumes from the latest checkpoint under `output_dir`; the default is `None` = fresh start). Optimizer, scheduler, RNG and step state are all restored (`save_only_model: false`).

Easy path (Kaggle): set the Kaggle Secret `MR_MT_RESUME=auto` and re-push the same kernel — the entrypoint restores the highest Drive checkpoint that has the `_upload_complete` marker (a checkpoint whose upload was cut off by the kill is skipped) and passes the flag for you.

Manual path:

1. Checkpoints land every 100 steps under the session's `output_dir` AND are streamed to Drive in the background (live mirror). After a 12h timeout, start a new session on the same account.

1. Re-attach the `HF_TOKEN` secret / datasets, reinstall the session's env, then

pull the latest confirmed checkpoint back onto the VM:

```
rclone lsf --dirs-only gdrive:mr-mt-edu-2026/bodhan-qlora/checkpoints
rclone lsf --files-only gdrive:mr-mt-edu-2026/bodhan-qlora/checkpoints/checkpoint-600 | grep _upl
rclone copy gdrive:mr-mt-edu-2026/bodhan-qlora/checkpoints/checkpoint-600 /kaggle/working/session
```

(or `snapshot_download` of the session's `hub.repo_id` if Hub push was on).

1. Re-launch with the flag pointing at the pulled dir, e.g.

```
PYTHONPATH=src python -m mr_mt.train_bodhan_qlora --config configs/sessionA_bodhan_qlora.yaml --
resume_from_checkpoint /kaggle/working/sessionA/checkpoint-600 (Or bash scripts/run_train_sessionA.sh
--resume_from_checkpoint <dir> — the launcher forwards "$@" ). reports/experiments.csv gets a
continuation row via mr_mt.utils.log_experiment (do not overwrite the prior row).
```

Cross-session resume works through the Drive mirror by default (rclone). The Hub is an optional second channel requiring ALL of: `hub.push_to_hub: true` AND a real `hub.repo_id` in the session config (the shipped configs have `push_to_hub: false` and a placeholder `"your-hf-user/..."` `repo_id`) AND a **write-role** `HF_TOKEN` (read role suffices for downloads only — see `docs/SETUP.md`). If rclone is missing on the VM, the callback prints a loud `!!! CHECKPOINTS ARE NOT LEAVING THE VM !!!` warning at the first save — treat that as fatal and fix the `gdrive-creds` dataset before continuing. `checkpointing.adapter_only_copy: true` mirrors are NOT resumable (no optimizer/scheduler/RNG) — eval-only artifacts.

Checkpoint cadence

- `save_steps: 100`, `eval_steps: 100`, `logging_steps: 10` (`configs/base.yaml`).
- Never increase `save_steps` past 100 on Kaggle: a 12h kill must cost ≤ 1 interval.
- `save_total_limit: null` keeps every checkpoint on Drive; the callback prunes

local copies beyond the newest `checkpointing.keep_local` (default 2) once their upload is confirmed, so `/kaggle/working` stays under its ~20GB quota. Prune Drive copies only after `metrics.json` is written.

Checkpoint persistence (keep every checkpoint)

`/kaggle/working` is ephemeral and only saved on a successful run, so checkpoints must be copied off the VM:

1. **Live Drive mirror (active, background):** each account has a private

`gdrive-creds` Dataset with `rclone.conf`; `kaggle_env.activate()` installs the rclone binary and points `RCLONE_CONFIG` at it. `checkpointing.rclone_remote` in the session configs (e.g. `gdrive:mr-mt-edu-2026/bodhan-qlora/checkpoints`) makes `CheckpointMirrorCallback` rclone each `checkpoint-<step>` to Drive as it is saved — in the background (`Popen` , capped at `max_pending` concurrent, all uploads awaited at

`on_train_end`), so training never stalls on network I/O. A `_upload_complete` marker is touched in the remote dir after each verified upload; always resume from a checkpoint that has the marker. Default `adapter_only_copy: false` keeps **full** (resumeable) checkpoints; set `true` for small adapter-only (non-resumable) copies.

1. Post-run Drive sync (fallback/backfill): `scripts/pull_kaggle_output.ps1`

→ `scripts/sync_drive.ps1` copies the whole run tree to `gdrive:mr-mt-edu-2026/<run>/`.

1. HF Hub (optional, off by default): set `hub.push_to_hub: true` + a real

`hub.repo_id` + a **write-role** token; `hub.strategy: all_checkpoints` streams every checkpoint for cross-session resume.

Verified locally: the mirror callback uploads a checkpoint to `gdrive:mr-mt-edu-2026/.../checkpoint-<step>`.

What to monitor

- **TensorBoard** (`training.logging_dir`, synced per session to Drive): train loss trend,

eval loss at each 100-step gate, LR schedule decay toward the min-LR floor.

- `reports/experiments.csv`: one row per (re)launch — run name, config hash, steps,

final loss, Hub/Drive pointers.

- **Sanity gates before trusting loss** ("loss drops but inference unchanged" — see

APPROACH.md §6): (a) greedy-decode a known train sample — output must be near-verbatim; (b) **nonce overfit**: train a few steps on a fake proper-noun pair, confirm the nonce sticks, then restore. If either fails, stop — the loss is masking/output-shift, not learning.

- Watch for: loss NaN/spike (P100 fp16 → lower LR), eval loss diverging from train

(overfit on 8k — raise dropout / cut steps), Hub push failures (session has no Internet).

Drive / Kaggle pull flow

```
kaggle kernels output -> artifacts/ -> scripts/sync_drive.ps1 (rclone) -> gdrive:mr-mt-edu-2026
```

1. Pull the finished kernel's `/kaggle/working` output into local staging

(gitignored) from the repo root:

```
powershell -ExecutionPolicy Bypass -File scripts/pull_kaggle_output.ps1 -Slug <owner/slug> -Acct
```

Downloads into `D:\Assignment\marathi-mt-bodhan\artifacts` (override with `-OutDir`).

1. Sync staging to Drive:

```
powershell -ExecutionPolicy Bypass -File scripts/sync_drive.ps1 [-DryRun]
```

(`make sync` does the same.) It runs `rclone copy artifacts/ -> gdrive:mr-mt-edu-2026/`; the `rclone` binary path is overridable via `scripts/rclone_path.txt`. `sync_drive.ps1` only uploads local files — it does NOT download from Kaggle.

1. Pulling back: download the session folder, place checkpoints under `models/` (gitignored)

or point `--adapter` at the Hub repo, then `make eval` / `make figures`.

LEAKAGE_CHECKLIST

Decontamination checklist — source: `reports/LEAKAGE_CHECKLIST.md`

LEAKAGE CHECKLIST — train/test decontamination

Status: pending — populate after the first successful train/eval cycle. The `prepare.py` decontamination gate (exact + normalized fuzzy overlap vs IN22-Gen/FLORES) must print zero leakage before training launches.

Eval sets (`ai4bharat/IN22-Gen` , `facebook/flores` devtest) are **held out, never trained on**. Every gate below must Pass before `make eval` numbers are reported. Tooling: `src/mr_mt/data/decontaminate.py` (`pair_hash` , `dedup_against` , `near_dup_filter` , `normalize`).

TODO(results): fill Result + Artifact per row after `make data` / `make eval`.

ID	Gate	Pass criterion	Result	Artifact
L1	Test pair-hashes vs train	0 exact (src,tgt) hash matches between train and IN22-Gen/FLORES	TODO	reports/decontam_11.json
L2	FLORES exact match	0 FLORES devtest pairs verbatim in train	TODO	reports/decontam_12.json
L3	IN22-Gen exact match	0 IN22-Gen pairs verbatim in train	TODO	reports/decontam_13.json
L4	FLORES src-only near-dup	0 train rows with src similarity > 0.9 to any FLORES src	TODO	reports/decontam_14.json
L5	IN22-Gen src-only near-dup	0 train rows with src similarity > 0.9 to any IN22-Gen src	TODO	reports/decontam_15.json
L6	Length filters	All train rows within min_words 1 – max_words 100 (configs/base.yaml1)	TODO	data/processed/ build log
L7	Date/domain provenance	Train slice is Education_v2 HIN-MAR (education-domain); rationale recorded in APPROACH.md §4 (not BPCC/Samanantar)	TODO	APPROACH.md §4
L8	No test in train dir	data/processed/train.jsonl contains no rows sourced from eval downloads; raw eval sets live outside train raw dir	TODO	data/raw/ listing
L9	Normalize-before-score	Scoring ran on normalized text (normalize(text, lang)); raw + normalized refs archived	TODO	reports/predictions/*.refs.txt
L10	Repro manifest	make data seed (42), dataset revisions, and removed-row counts (dedup_against / near_dup_filter tallies) recorded	TODO	reports/experiments.csv + build log

Engineering log: the four failures that shaped this repo

The brief scores effort, judgement and problem-solving rather than metrics, so the debugging trail is documented here instead of hidden in commit history. Each failure was diagnosed to its root cause, fixed in the code (not worked around), and re-run end-to-end on Kaggle.

- **QLoRA OOM at 10.5 GiB.** PEFT's stock `prepare_model_for_kbit_training` upcasts every low-precision tensor to FP32, including Gemma's multi-gigabyte embedding table, which attempted a 10.5 GiB allocation on a 16 GiB T4 before step 1. Replaced with an architecture-aware local helper that upcasts only 1-D norm vectors and keeps large matrices in the compute dtype.
- **assistant-only loss RuntimeError.** TRL aborts when a row has no assistant mask because the gated chat template emits no generation markers. Disabled assistant-only loss for these runs, and recorded the semantic trade-off (full-sequence loss) rather than silently dropping it.
- **GradScaler / BF16 mismatch.** On a T4, `torch.cuda.is_bf16_supported()` returns True through sm75 emulation, so the model loaded in BF16 while the trainer ran the FP16 scaler, crashing on the first gradient unscale with `NotImplementedError: _amp_foreach_non_finite_check_and_unscale_cuda not implemented for 'BFloat16'`. Fixed by letting the explicit `training.fp16/bf16` flags override the GPU capability probe instead of trusting it.

- **FP16 master weights.** Forcing FP16 was not sufficient either: the AMP contract requires FP32 master weights with autocast performing the casting, so the next run failed with `ValueError: Attempting to unscale FP16 gradients`. Trainable LoRA parameters are now upcast to FP32 *after* the adapters are constructed (they are created inside `SFTTrainer.__init__`, so aligning earlier would have missed all 100,999,168 of them), while the frozen 4-bit base stays low-precision to keep memory in budget.

All three sessions now complete, with a full resumable checkpoint saved and mirrored to Drive every 5 optimizer steps, each upload confirmed by an `_upload_complete` marker before local copies are pruned.